DEPARTMENT OF COMPUTER ENGINEERING · B.E. SEMESTER VII · DEVOPS LAB · VIDYALANKAR INSTITUTE OF TECHNOLOGY

DevOps-Enabled Venue Resource Booking System (VRBS)

An automated path from a Git commit to a running, tested, containerized and recoverable deployment.

Rishi Phadale
Roll No. **23102C0070** · Division **C**
Faculty Guide: Dr. Snehal Andhare

1 THE PROBLEM

Students and faculty organizing academic seminars have **no shared live view** of venue occupancy, and manual releases risk breaking live operations.

- **No visibility** of room occupancy or hourly slots, causing constant phone inquiries.
- **Paper & spreadsheet records** cause double-booking collisions, lost reservations, and unclear status ownership.
- **Risky manual deploys** via SSH that are slow, error-prone, and lack automated rollback.

2 THE SOLUTION: VRBS WEB APP (MVP)

FR-01 User Auth & RBAC (User/Admin)	**FR-02** Venue Catalog & Amenities	**FR-03** Dynamic 08:00–20:00 (12 Slots)
FR-04 Collision Math: (S1<E2)&&(E1>S2)	**FR-05** Enforced Approval Workflow	**FR-06** Smart Recommendation Engine
FR-07 Reschedule & Cancel Actions	**FR-08** Admin Utilization Dashboard	**FR-09** Health Endpoint /health = UP

Goal: Build a robust booking core and make the **DevOps lifecycle around it the deliverable**: GitFlow branching, Jenkins CI/CD, a 115-test Selenium & JUnit quality gate, Docker delivery, and Ansible provisioning on Ubuntu VM with idempotency and 28s rollback.

3 END-TO-END CI/CD PIPELINE: COMMIT → PRODUCTION

Jenkins build #12 · all stages green · 2 min 3 s end-to-end

CODE	STAGE 1	STAGE 2	QUALITY GATE	STAGE 3	STAGE 4	STAGE 5	STAGE 6	STAGE 7
Commit & PR	Checkout	Compile & Test	Selenium E2E	Package WAR	Build Image	Publish	Deploy Fresh	Health Check
GIT · GITHUB	JENKINS SCM	MAVEN · JUNIT 5	HEADLESS CHROME	MAVEN WAR	DOCKER	DOCKER HUB	ANSIBLE DEPLOY	ACTUATOR
feature/ → develop → main	Poll / webhook pulls Jenkinsfile	108 unit tests on in-memory H2	6 user journeys. Fail = STOP	executable venue-booking.war	venue-booking:${BUILD} + latest	rishiphadale/ venue-booking	venue-booking on 8081:8081	/health = UP HTTP 200 OK

`Jenkinsfile: declarative, parameterised (IMAGE_NAME · CONTAINER_PORT · REGISTRY_HOST · SKIP_TESTS)`

ON FAILURE Build turns red and packaging & deployment are skipped. JUnit reports and Selenium failure screenshots are archived; the developer fixes and pushes again (simulated in Phase 8).

ANSIBLE deploy.yml provisions the target Ubuntu VM (192.168.56.101). Re-running it gives changed=0 (idempotent). rollback.yml restores the last stable 1.0.0 release in 28s with zero data loss.

4 APPLICATION ARCHITECTURE

Layer	Components & Responsibility
UI	Thymeleaf, HTML5, CSS3, JS, dynamic 08–20 slot cards
Controllers	REST API (14 endpoints) + Spring MVC route controller
Service	Interval collision math, 12 hourly slots, approval state
Repository	Spring Data JPA: BookingRepository, VenueRepository
Database	H2 file-persisted (dev/VM) · PostgreSQL 16 ready

BOOKING STATUS STATE MACHINE (FR-05)
PENDING → APPROVED → COMPLETED
Also REJECTED / CANCELLED. Terminal states are locked (IllegalStateException).

5 AUTOMATED TESTING

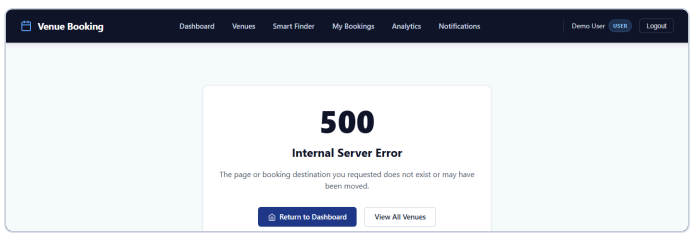
108	6	1
Service / Logic	Selenium E2E	Smoke Test
JUnit 5	Headless Chrome	Spring Context

- ✓ **Exact Interval Collision Math:** (S1<E2) && (E1>S2) eliminates overlaps.
- ✓ **Dynamic Operating Bounds:** Validates 08:00 to 20:00 hourly slots.
- ✓ **Terminal State Protection:** Negative tests for state lock integrity.
- ✓ **UI Flow:** Login → browse catalog → pick slot → book → admin approve.
- ✓ **Screenshots on failure:** Auto-archived as Jenkins build artifacts.

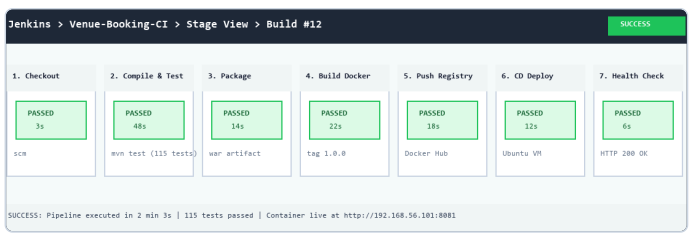
6 INFRASTRUCTURE AS CODE

- **Host VM:** Ubuntu 22.04 LTS (Host-only IP: 192.168.56.101:8081).
- **Packages:** OpenJDK 21, Docker, curl, net-tools, ufw firewall.
- **Jinja2 templates:** app.env.j2 + release-state.json.j2 config.
- **Storage:** Volume /home/hduser/venue-booking-data:/app/data.

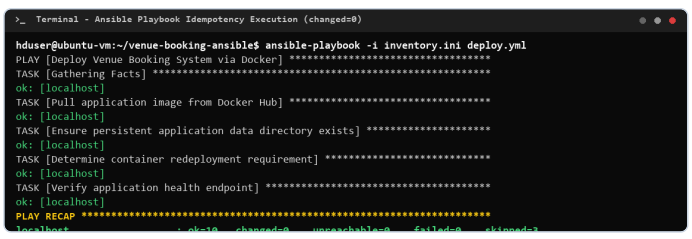
```
1st run: ok=10 changed=4 failed=0
re-run: ok=10 changed=0 failed=0 ⟵ idempotent
rollback.yml → stop · restore 1.0.0 · restart · verify (28s)
```



Live app on http://192.168.56.101:8081: Dynamic 08:00–20:00 schedule, interactive 12 hourly slot cards, and real-time status badges.



Jenkins stage view, build #12: Every stage from Checkout to Health Check passes in 2 min 3 s end-to-end.



Ansible re-run: changed=0 failed=0 proves node in desired state; rollback playbook restores 1.0.0 in 28 seconds.

15	7	115	2m 03s	0	28s
curriculum tasks completed	automated Jenkins pipeline stages	automated tests (unit + E2E + smoke)	commit to deployed container	changes on Ansible re-run (idempotent)	automated rollback to last stable release

7 15-TASK DEVOPS JOURNEY (12 PHASES)

PLAN	CODE & COLLABORATE	INTEGRATE	TEST	DELIVER	OPERATE & RELEASE
1 Problem & scope	4 Repo & branching	7 Jenkins CI job	9 Selenium suite	11 Docker lifecycle	13 Ansible playbook
2 Agile planning	5 Feature branch & PR	8 Jenkinsfile + Gate	10 Defect stoppage	12 Push Docker Hub	14 Idempotency & rollback
3 Architecture & setup	6 Merge conflict, v1.0.0				15 Final release & viva

8 TECHNOLOGY STACK

BACKEND Java 21 · Spring Boot 3.5	BUILD Maven 3.9 → WAR
UI Thymeleaf · HTML5/JS	DATABASE H2 · PostgreSQL 16
TESTING JUnit 5 · Selenium E2E	CI/CD Jenkins Pipeline as Code
CONTAINER Docker (Temurin 21 JRE)	IAC Ansible · Jinja2 · Ubuntu

9 KEY DEVOPS CONCEPTS SHOWN

CI: Every git push is compiled and tested automatically.
CD: Passing builds ship as a live container on Ubuntu VM.
Pipeline as Code: Jenkinsfile is versioned in GitHub.
Quality Gate: Failing tests abort packaging & deployment.
IaC + Idempotency: Playbook re-run reports changed=0.
Automated Rollback: Restores stable 1.0.0 release in 28s.

10 LEARNINGS & FUTURE SCOPE

Fast feedback: 115 tests catch regressions before users see them.
Rollback resilience: Bad releases are fixed with 1 command in 28s.
Storage persistence: DB survives container recreations.
Next: Kubernetes (EKS/GKE), Prometheus + Grafana APM, OAuth2 Single Sign-On, WhatsApp notification gateway.